Mise en place d'un pipeline CI/CD DevOps complet sur AWS EKS

(Jenkins, SonarQube, Nexus, Terraform, Monitoring)

Encadrante : Pr. Thouraya Louati
Réalisé par : Amine Boulila
Classe : IGL5– Faculté des Sciences de Tunis, Université Tunis El Manar
Année universitaire : 2026-2027

Sommaire et architecture globale

Chaîne CI/CD : du commit à l'application monitorée



Webhook GitHub → Jenkins → build / qualité / artefact → image Docker → Terraform / EKS → Kubernetes → monitoring

Entrées
GitHub • branche feature/amine • webhook

Sorties
LoadBalancer AWS • Grafana • Alertmanager

Sommaire de la soutenance

Technologies

Git / GitHub

Jenkins

Maven & tests

SonarQube

Nexus

Docker / Docker Hub

AWS + Terraform

Déploiement et bilan

Kubernetes / EKS

Monitoring

Webhook & ngrok

Pipeline Destroy

Résultat final

Difficultés et améliorations

Conclusion

Dépôt, branche et commandes

Référentiel

Dépôt : github.com/amine-boulila/IGL5-G8-StudentManagement

Branche CI/CD : feature/amine

Application Spring Boot + MySQL

Webhook GitHub → Jenkins pour chaque commit

Commandes / installation

sudo apt-get update

sudo apt-get install -y git

git clone <URL>

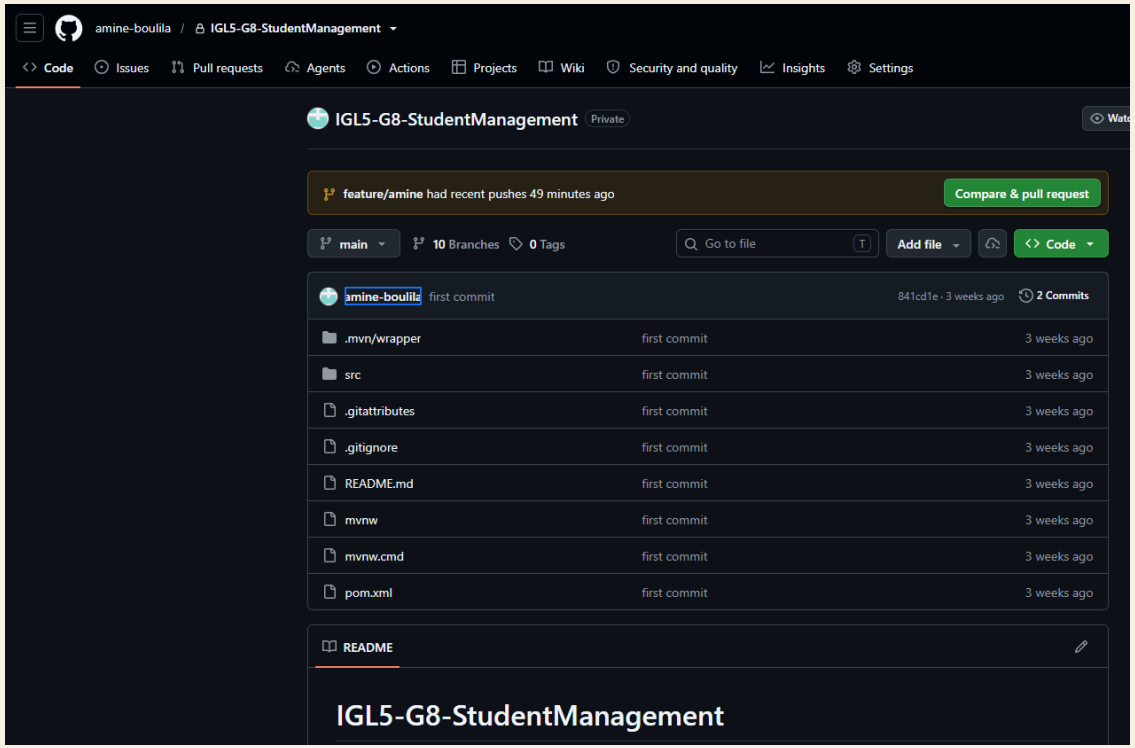
git checkout feature/amine

git add .

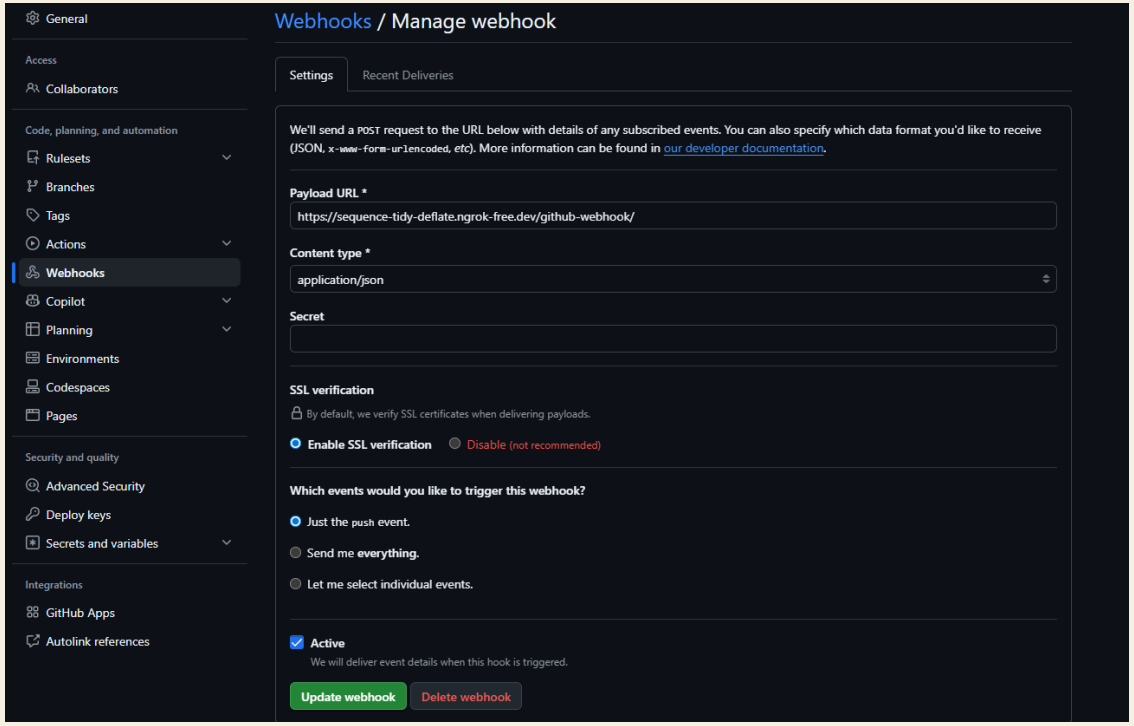
git commit -m "..."

git push origin feature/amine

Preuve : dépôt et webhook



Dépôt GitHub



Webhook GitHub

Installation, plugins, outils et credentials

Installation / outils

docker volume create jenkins_home

docker run -d --name jenkins -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts-jdk21

Plugins : Pipeline, Git, Credentials, JUnit, JaCoCo, SonarQube, Docker, AWS / Kubernetes

Outils globaux : JDK21 et Maven3

Agent : any

Credentials IDs

github-creds

aws-lab-creds (fichier)

sonarqube-token

nexus-creds

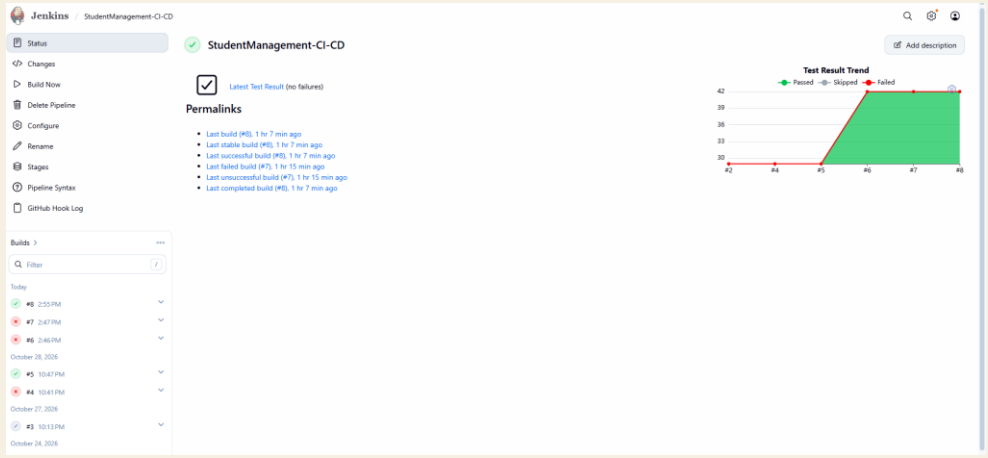
dockerhub-creds

smtp-app-password

Variables environment : SONAR_URL, NEXUS_URL, SONAR_PROJECT_KEY, AWS_DEFAULT_REGION, EKS_CLUSTER_NAME, EKS_ROLE_NAME

Pipeline déclaratif : stages et preuves

```
pipeline {
  agent any
  tools { jdk 'JDK21'; maven 'Maven3' }
  stages {
    stage('Checkout Git') { ... }
    stage('MVN Clean') { sh 'mvn clean' }
    stage('Artifact Construction') { ... }
    stage('Unit Tests') { ... }
    stage('MVN SonarQube') { ... }
    stage('Publish to Nexus') { ... }
    stage('Building Our Image') { ... }
    stage('Deploy Our Image') { ... }
  }
}
```

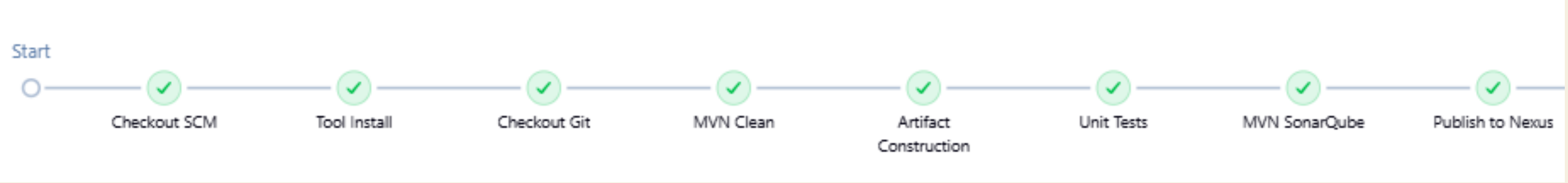


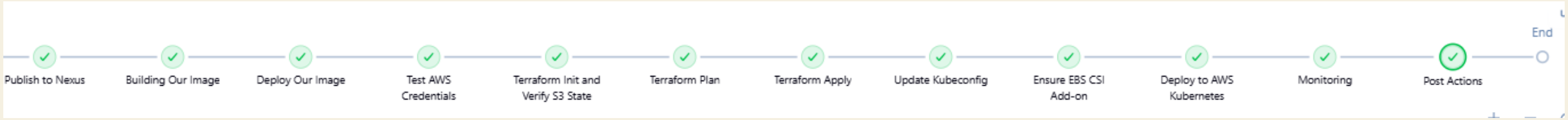
Dashboard Jenkins



Credentials Jenkins

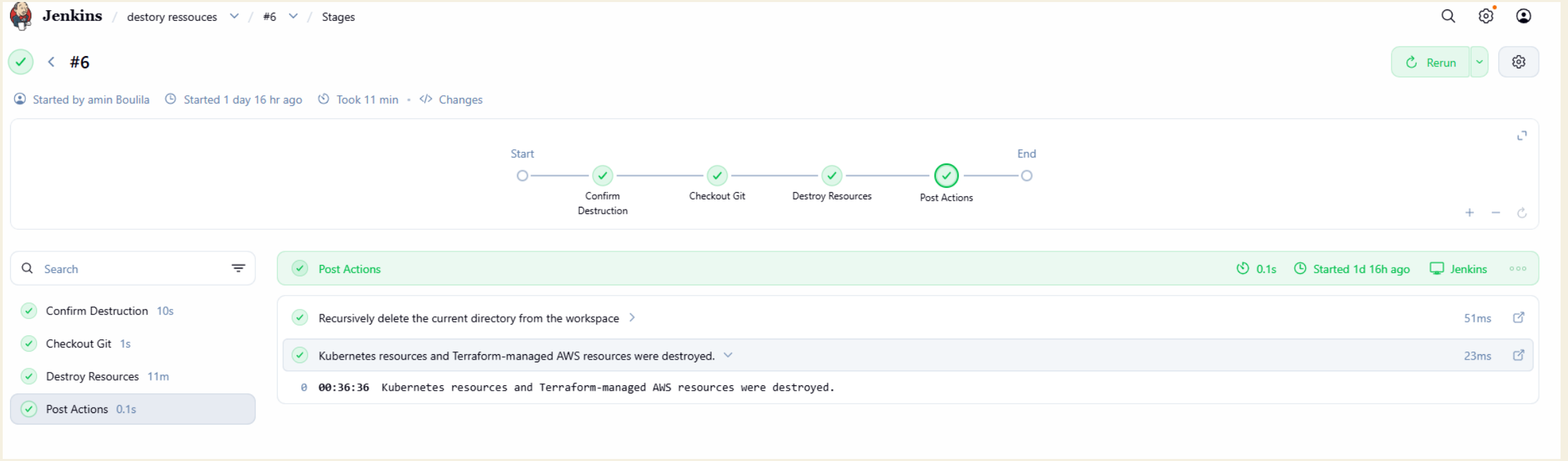
Pipeline Deploy : déploiement réussi





Pipeline Jenkins Deploy — stages exécutés avec succès

Pipeline Destroy : nettoyage contrôlé



Pipeline Jenkins Destroy — suppression contrôlée

Installation, construction et qualité de test

Installation / commandes

sudo apt-get update

sudo apt-get install -y openjdk-21-jdk maven

mvn clean

mvn package -DskipTests

mvn verify

test -s target/site/jacoco/jacoco.xml

Rapports JUnit : **/target/surefire-reports/*.xml

Configuration

Java demandé dans Jenkins : JDK21

Version projet dans pom.xml : Java 17

JaCoCo : rapport en phase verify

JUnit : publication Jenkins via junit()

Extrait : stage Unit Tests et JaCoCo

```
                                        stage('Unit Tests') {
    steps {
        sh '''
            mvn verify
            test -s target/site/jacoco/jacoco.xml
        '''
    }
    post {
        always {
            junit allowEmptyResults: true,
                testResults: '**/target/surefire-reports/*.xml'
        }
    }
}
```

Le fichier jacoco.xml devient l'entrée de SonarQube ; les XML Surefire sont publiés par Jenkins.

Installation, token et analyse de couverture

Installation / configuration

docker run -d --name sonarqube -p 9000:9000 sonarqube:lts-community

URL configurée : http://host.docker.internal:9000

Projet : IGL5-G8-StudentManagement

Credential Jenkins : sonarqube-token

Secret transmis via SONAR_TOKEN

Commande du stage

mvn org.sonarsource.scanner.maven:sonar-maven-plugin:5.2.0.4988:sonar

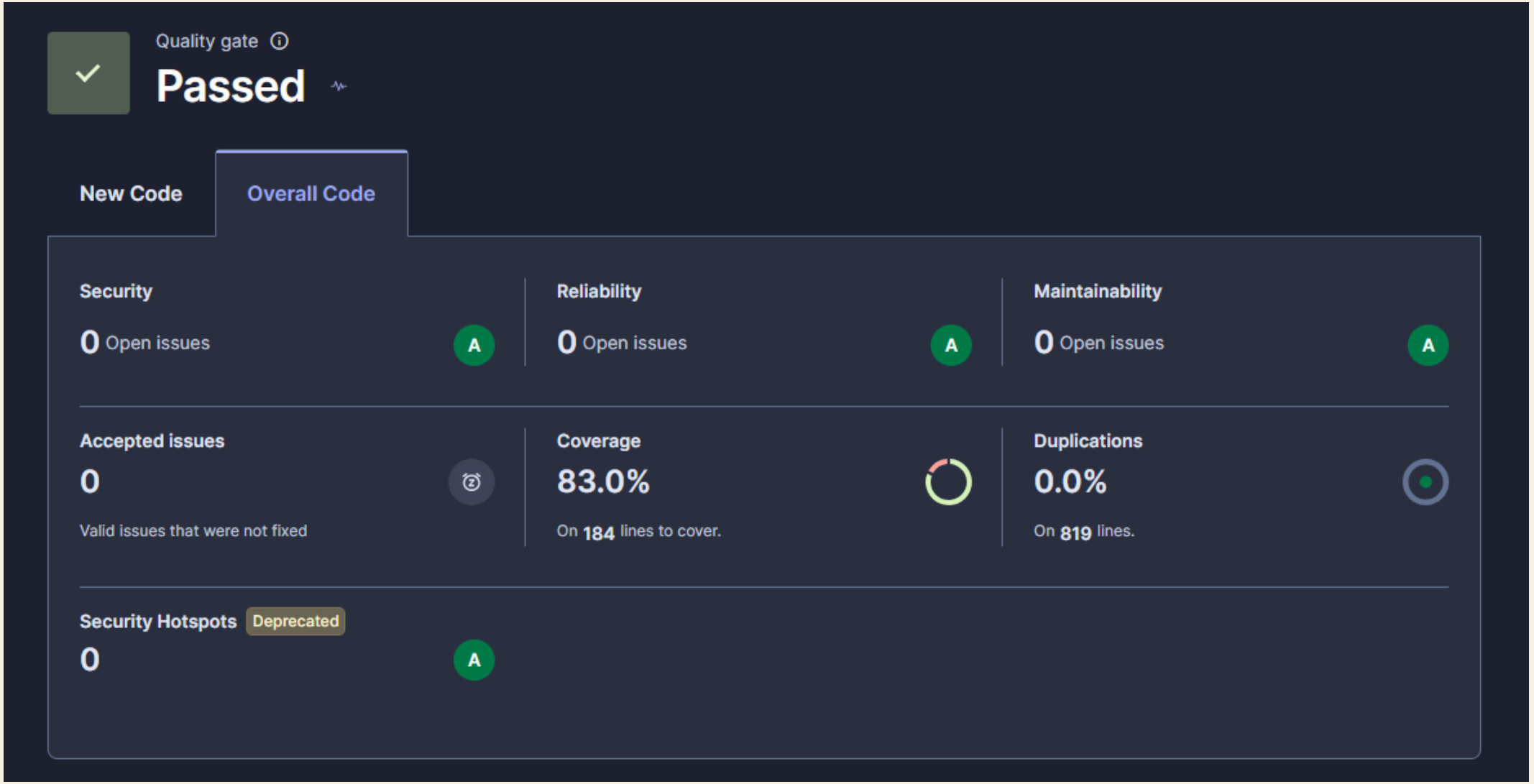
-Dsonar.host.url="$SONAR_URL"

-Dsonar.projectKey="$SONAR_PROJECT_KEY"

-Dsonar.coverage.jacoco.xmlReportPaths="target/site/jacoco/jacoco.xml"

-Dsonar.token=********

Preuve : dashboard de qualité



Dashboard SonarQube du projet

Repositories, credentials et publication Maven

Installation / repositories

docker run -d --name nexus -p 8081:8081 sonatype/nexus3

URL : http://host.docker.internal:8081

maven-snapshots

maven-releases

Credential Jenkins : nexus-creds

Commandes / sécurité

settings.xml est généré temporairement par le pipeline

mvn -s jenkins-settings.xml deploy

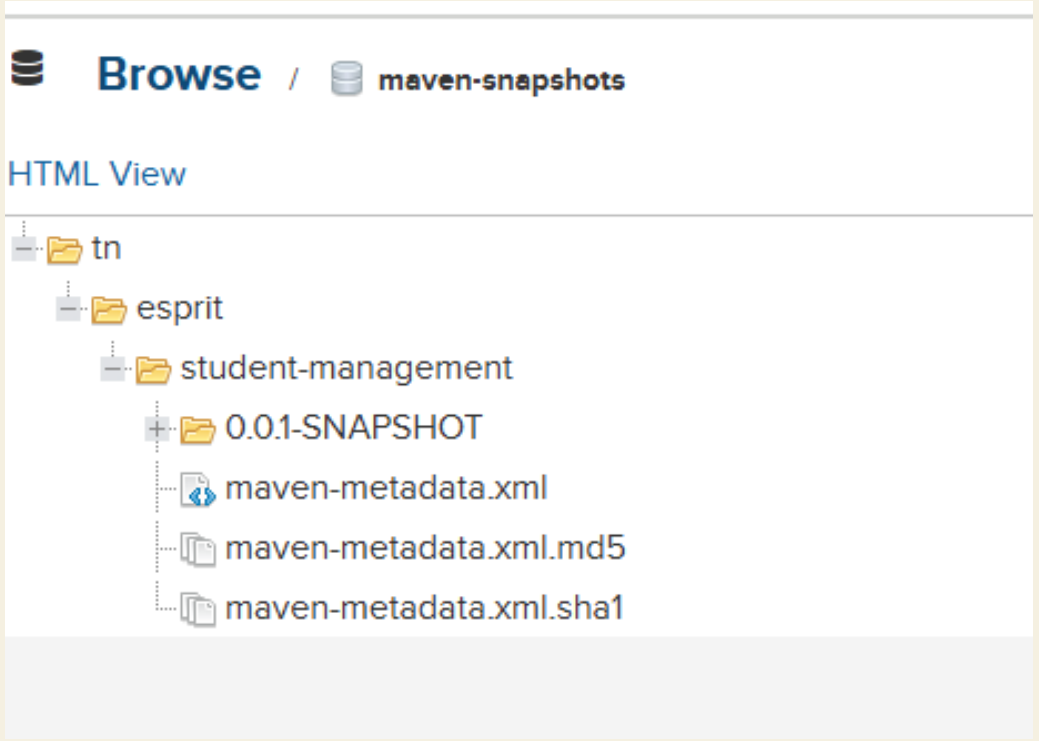
-DskipTests

Publication vers maven-snapshots ou maven-releases

Le fichier est supprimé dans finally

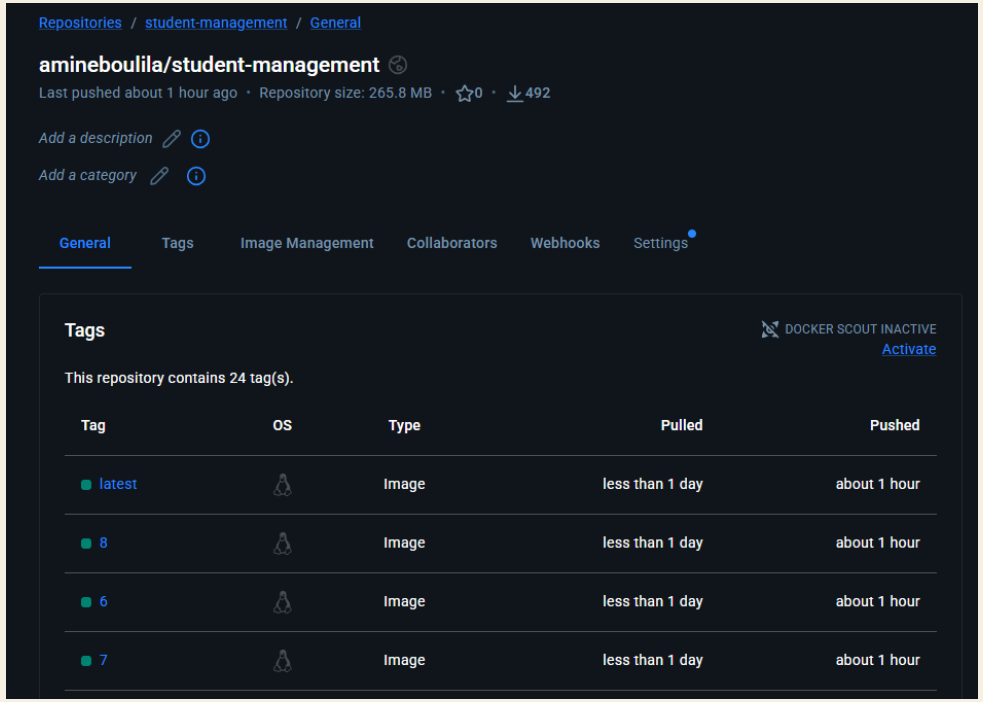
Preuve et extrait de publication

```
   writeFile file: 'jenkins-settings.xml', text: """<settings>
  <servers>
    <server><id>nexus-snapshots</id>
      <username>${xmlEscape(env.NEXUS_USER)}</username>
      <password>${xmlEscape(env.NEXUS_PASSWORD)}</password>
    </server>
  </servers>
</settings>"""
mvn -s jenkins-settings.xml deploy -DskipTests
rm -f jenkins-settings.xml
```



Repositories Nexus

Dockerfile, build, tags et push




```
docker build -t student-management:${BUILD_NUMBER} .
docker tag student-management:${BUILD_NUMBER} $DOCKERHUB_USER/student-management:${BUILD_NUMBER}
docker tag student-management:${BUILD_NUMBER} $DOCKERHUB_USER/student-management:latest
echo "$DOCKERHUB_TOKEN" | docker login --username "$DOCKERHUB_USER" --password-stdin
docker push $DOCKERHUB_USER/student-management:${BUILD_NUMBER}
docker push $DOCKERHUB_USER/student-management:latest
docker logoutaa
```

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 8089
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Repository Docker Hub

AWS Academy, identité et état Terraform

Contexte AWS

Région : us-east-1

Rôle : LabRole

Cluster : student-management-eks

Credential Jenkins : aws-lab-creds (fichier)

EBS CSI driver activé pour les volumes gp3

Installation AWS CLI : sudo apt-get update && sudo apt-get install -y awscli

Commandes

aws sts get-caller-identity

aws eks update-kubeconfig --region "$AWS_DEFAULT_REGION" --name "$EKS_CLUSTER_NAME"

Backend S3 : bucket student-management-tfstate-amine-2026

Clé : student-management/terraform.tfstate

Chiffrement : encrypt = true

Preuve : nœuds EC2

Cluster: Nodes (2)

A node is a worker machine in Kubernetes. Learn more

Last updated
October 30, 2026, 16:55 (UTC+01:00)

View details

Filter Nodes by name

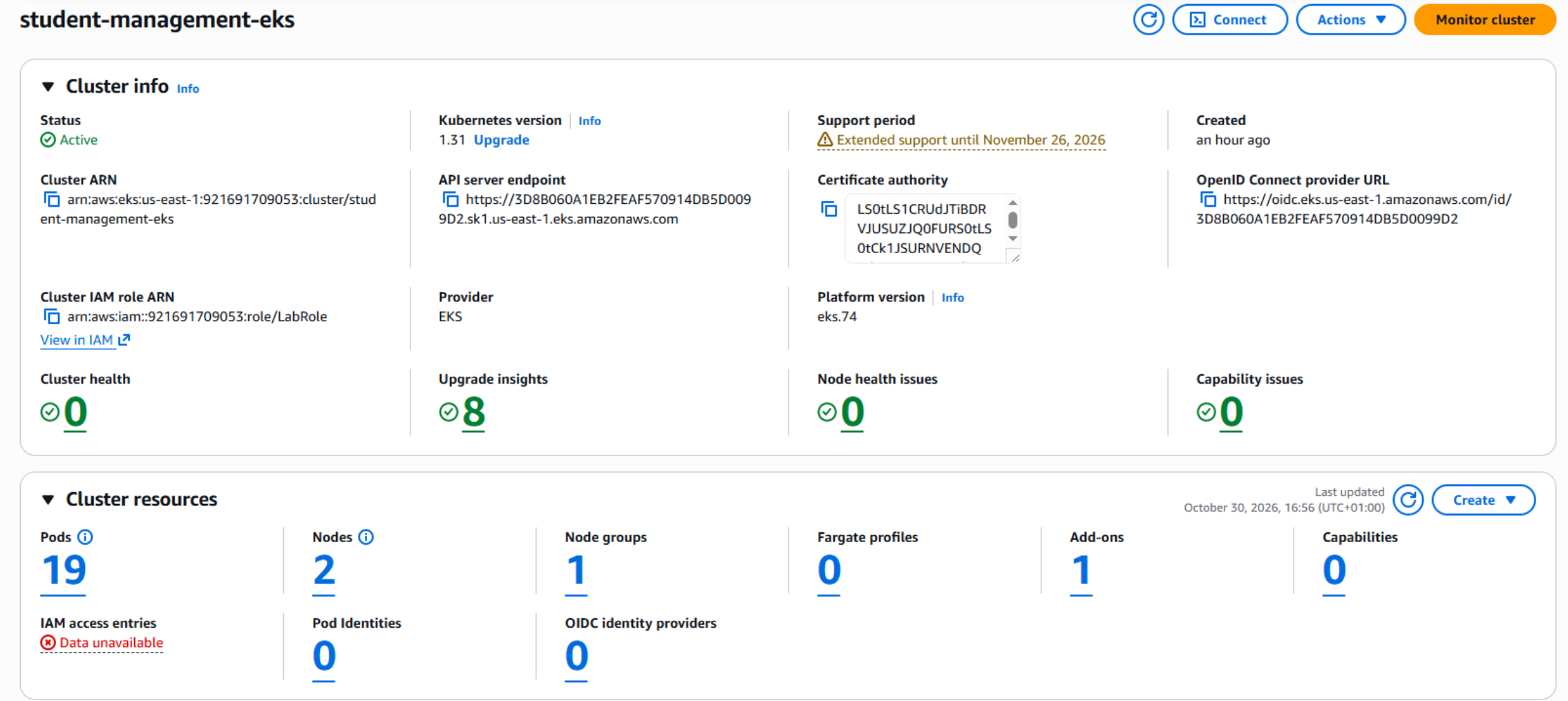
Advanced query filters

1

Node name	Instance type	Capacity type	Compute	Managed by	Created	Status	CPU usage	Memory usage	Ephemeral storage usage
ip-10-0-1-232.ec2.internal	t3.medium	On-Demand	Node group	student-management-eks-nodes	an hour ago	Ready	31%	40%	15%
ip-10-0-2-233.ec2.internal	t3.medium	On-Demand	Node group	student-management-eks-nodes	an hour ago	Ready	37%	45%	15%

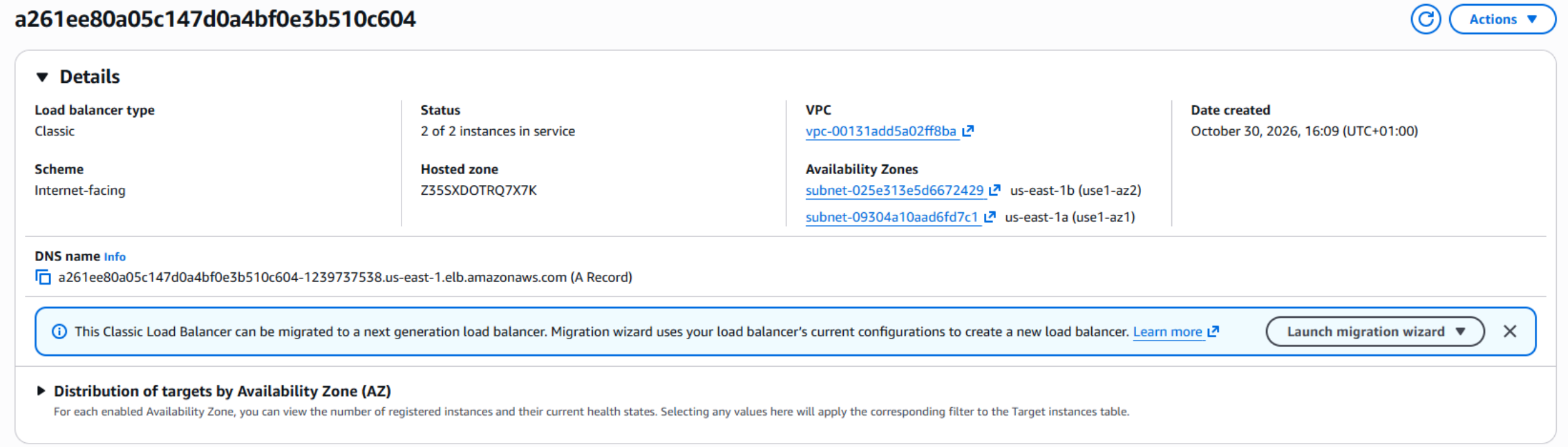
Nœuds EC2 du cluster

Preuve : cluster EKS



Cluster EKS student-management-eks

Preuve : Load Balancer



Load Balancer AWS

Installation, structure et cycle d'infrastructure

Installation / fichiers

Installation : wget -O- https://apt.releases.hashicorp.com/gpg | gpg --dearmor | sudo tee /usr/share/keyrings/hashicorp-archive-keyring.gpg

sudo apt-get update && sudo apt-get install -y terraform

terraform/versions.tf : provider AWS ~> 5.0, Terraform >= 1.10.0

terraform/backend.tf : état distant S3

terraform/main.tf : VPC, subnets, EKS, node group, add-on

terraform/variables.tf et outputs.tf

Commandes exactes

terraform init -reconfigure -input=false

terraform validate

terraform state list

terraform plan -input=false ... -out=tfplan

terraform show -no-color tfplan

terraform apply -input=false tfplan

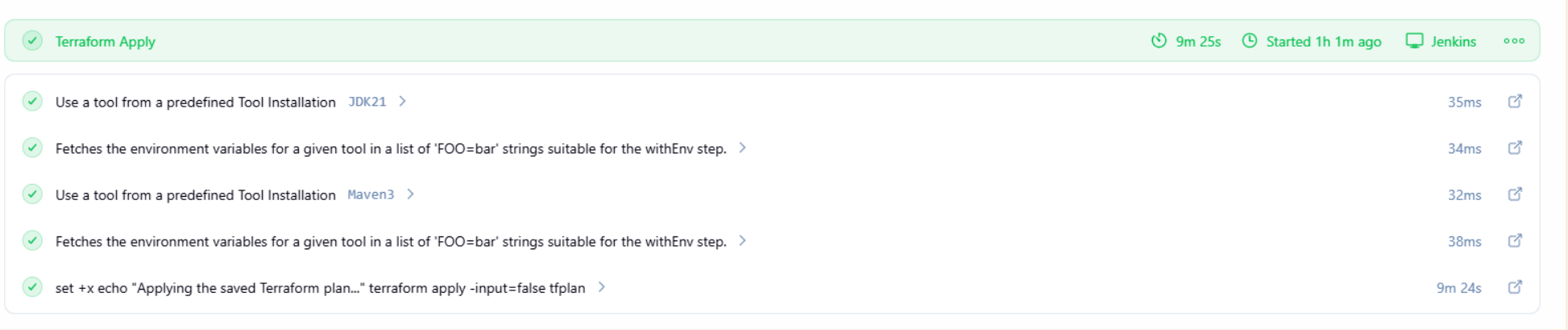
terraform destroy -input=false -auto-approve ...

Extrait : EKS et état distant

```
                              terraform {
  backend "s3" {
    bucket       = "student-management-tfstate-amine-2026"
    key          = "student-management/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    use_lockfile = true
  }
}

    subnet_ids              = aws_subnet.public[*].id
    endpoint_private_access = false
    endpoint_public_access  = true
  }

  tags = local.common_tags
}
```



Manifests, stockage et déploiement

Commandes

Installation kubectl : curl -LO https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl && sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

kubectl get nodes

kubectl apply -f kubernetes/mysql.yaml

kubectl rollout status deployment/mysql --timeout=300s

kubectl apply -f kubernetes/deployment.yaml

kubectl apply -f kubernetes/service.yaml

kubectl get deployments / pods / services

Architecture

MySQL : Deployment + ClusterIP + PVC 5Gi

StorageClass gp3 via ebs.csi.aws.com

Application : image amineboulila/student-management:latest

Service application : type LoadBalancer

Accès public via DNS du Load Balancer AWS

Extraits des manifests et vérification

```
apiVersion: v1
kind: Service
metadata:
  name: student-management
spec:
  type: LoadBalancer
  selector:
    app.kubernetes.io/name: student-management
  ports:
    - port: 80
      targetPort: httpa
```

```
kubectl get nodes
kubectl get deployments
kubectl get pods
kubectl get services

aws eks create-addon \
  --cluster-name "$EKS_CLUSTER_NAME" \
  --addon-name aws-ebs-csi-driver \
  --region "$AWS_DEFAULT_REGION"
```

```
PS E:\studies\igl5\devops\student-management\student-management> kubectl get nodes
NAME                          STATUS   ROLES    AGE   VERSION
ip-10-0-1-232.ec2.internal    Ready    <none>   58m   v1.31.14-eks-cfb47f5
ip-10-0-2-233.ec2.internal    Ready    <none>   58m   v1.31.14-eks-cfb47f5
PS E:\studies\igl5\devops\student-management\student-management> kubectl get pods -A
NAMESPACE     NAME                                                     READY   STATUS    RESTARTS   AGE
default       mysql-7dcfddc697-6lnd9                                   1/1     Running   0          57m
default       student-management-698c868996-mx9xl                      1/1     Running   0          57m
kube-system   aws-node-gt7zn                                           2/2     Running   0          59m
kube-system   aws-node-zvqck                                           2/2     Running   0          59m
kube-system   coredns-5bb66f7889-gvkw6                                 1/1     Running   0          60m
kube-system   coredns-5bb66f7889-vm97s                                 1/1     Running   0          60m
kube-system   ebs-csi-controller-685d4dd5d5-rdkw6                      6/6     Running   0          58m
kube-system   ebs-csi-controller-685d4dd5d5-tqvj9                      6/6     Running   0          58m
kube-system   ebs-csi-node-9fcck                                       3/3     Running   0          58m
kube-system   ebs-csi-node-m5jld                                       3/3     Running   0          58m
kube-system   kube-proxy-h2lmb                                         1/1     Running   0          59m
kube-system   kube-proxy-sz52z                                         1/1     Running   0          59m
monitoring    alertmanager-monitoring-kube-prometheus-alertmanager-0   2/2     Running   0          54m
monitoring    monitoring-grafana-684b6c8f75-hzqdq                      3/3     Running   0          54m
monitoring    monitoring-kube-prometheus-operator-855777dd99-4f6b5     1/1     Running   0          54m
monitoring    monitoring-kube-state-metrics-75d7ddcc9c-jshpd           1/1     Running   0          54m
monitoring    monitoring-prometheus-node-exporter-vn8zb                1/1     Running   0          54m
monitoring    monitoring-prometheus-node-exporter-zg6gq                1/1     Running   0          54m
monitoring    prometheus-monitoring-kube-prometheus-prometheus-0       2/2     Running   0          54m
```

```
PS E:\studies\igl5\devops\student-management\student-management> kubectl get svc
NAME                TYPE           CLUSTER-IP       EXTERNAL-IP                                                              PORT(S)        AG
E
kubernetes          ClusterIP      172.20.0.1       <none>                                                                   443/TCP        62
m
mysql               ClusterIP      172.20.109.117   <none>                                                                   3306/TCP       58
m
student-management  LoadBalancer   172.20.98.61     a261ee80a05c147d0a4bf0e3b510c604-1239737538.us-east-1.elb.amazonaws.com   80:30712/TCP   58
m
```

Services Kubernetes

Installation Helm et configuration des alertes

Stack déployée

Prometheus : rétention 3 jours

Grafana : dashboards

Alertmanager : receiver email

SMTP Gmail : port 587 / TLS

Mot de passe SMTP : ********

```
      helm repo add prometheus-community \
  https://prometheus-community.github.io/helm-charts
helm repo update
helm upgrade --install monitoring \
  prometheus-community/kube-prometheus-stack \
  -n monitoring --create-namespace \
  -f monitoring-values.yaml \
  --set-string "alertmanager.config.global.smtp_auth_password=********" \
  --timeout 10m
kubectl get pods -n monitoring
kubectl get services -n monitoring
```

Installation / accès

curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

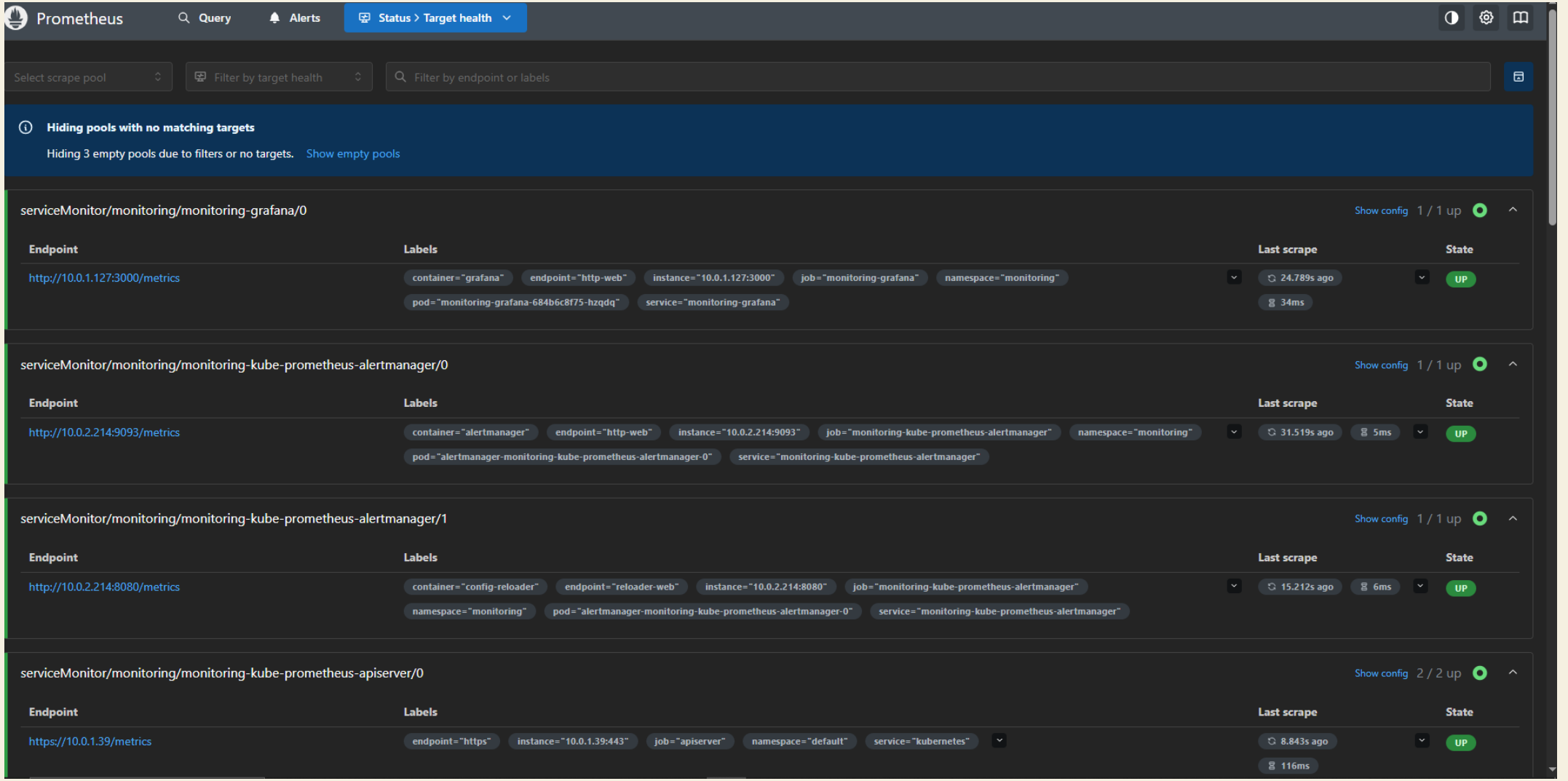
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts

kubectl port-forward -n monitoring svc/monitoring-kube-prometheus-prometheus 9090:9090

kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80

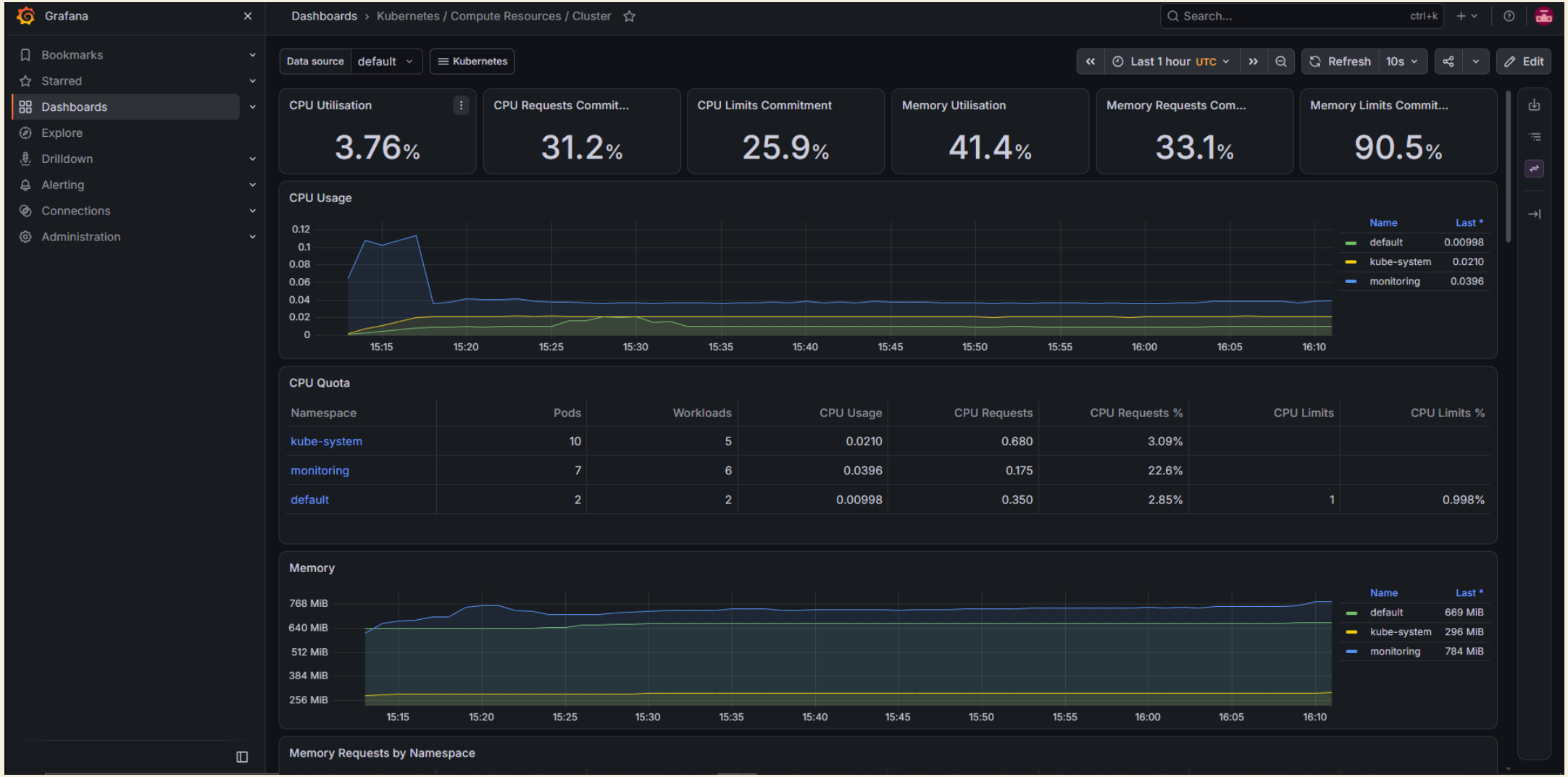
kubectl port-forward -n monitoring svc/monitoring-kube-prometheus-alertmanager 9093:9093

Preuve : cibles Prometheus



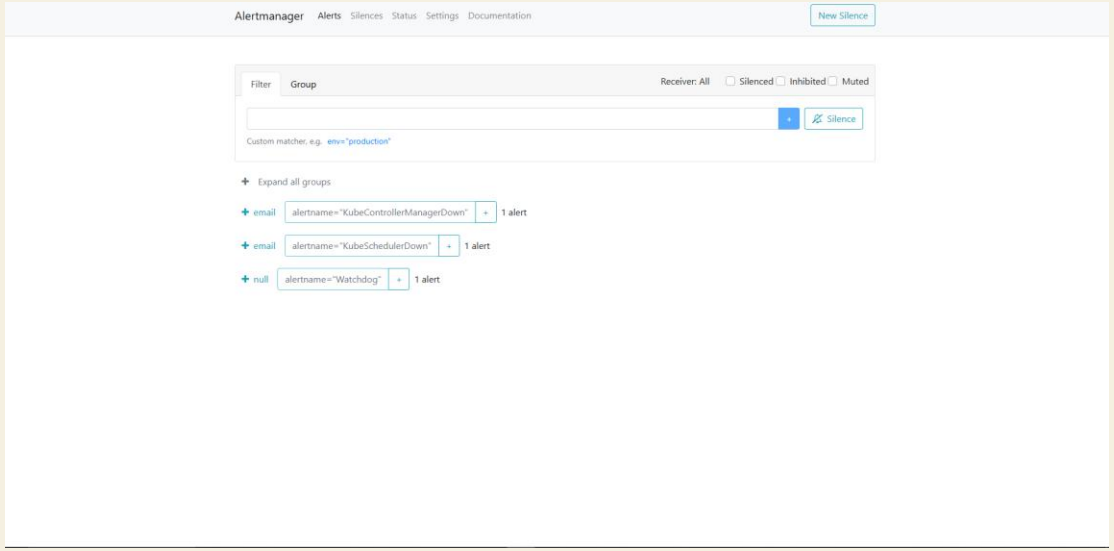
Targets Prometheus

Preuve : dashboard Grafana

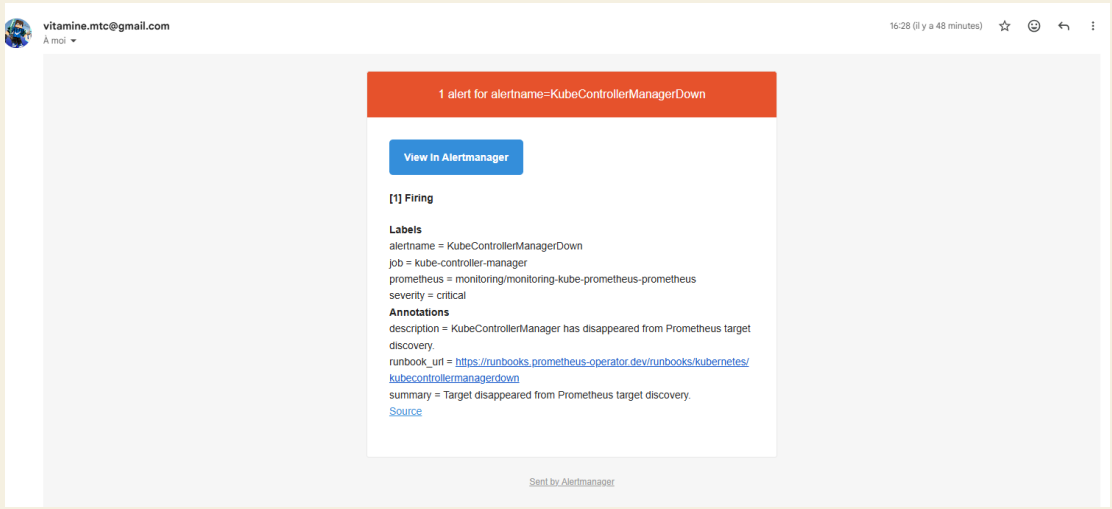


Dashboard Grafana

Preuves : Alertmanager et email



Alertmanager



Email d'alerte

Déclenchement automatique

Commande présente dans le dépôt

winget install Ngrok.Ngrok

ngrok config add-authtoken ********

ngrok http --url=yeasty-brainsickly-lela.ngrok-free.dev 8080

Le tunnel expose Jenkins local

URL de callback configurée dans GitHub

Flux

Commit sur feature/amine

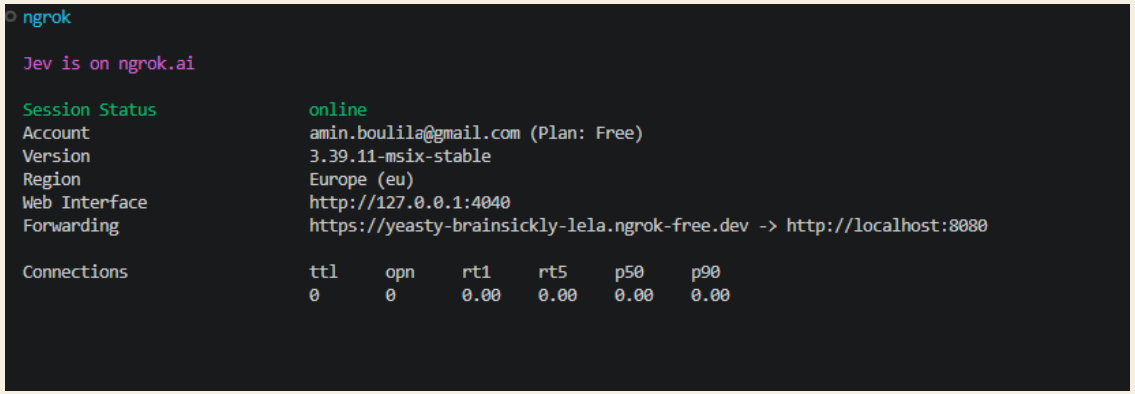
GitHub envoie push webhook

ngrok relaie vers Jenkins

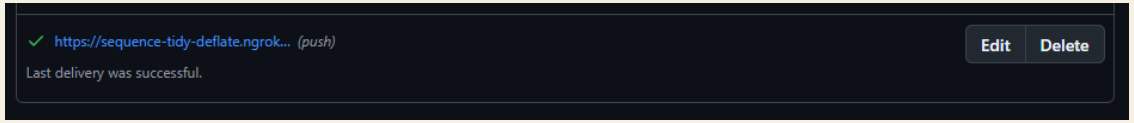
Jenkins lance le pipeline

Vérification : Build automatique à chaque commit

Preuves : tunnel et déclenchement Jenkins



Dashboard ngrok

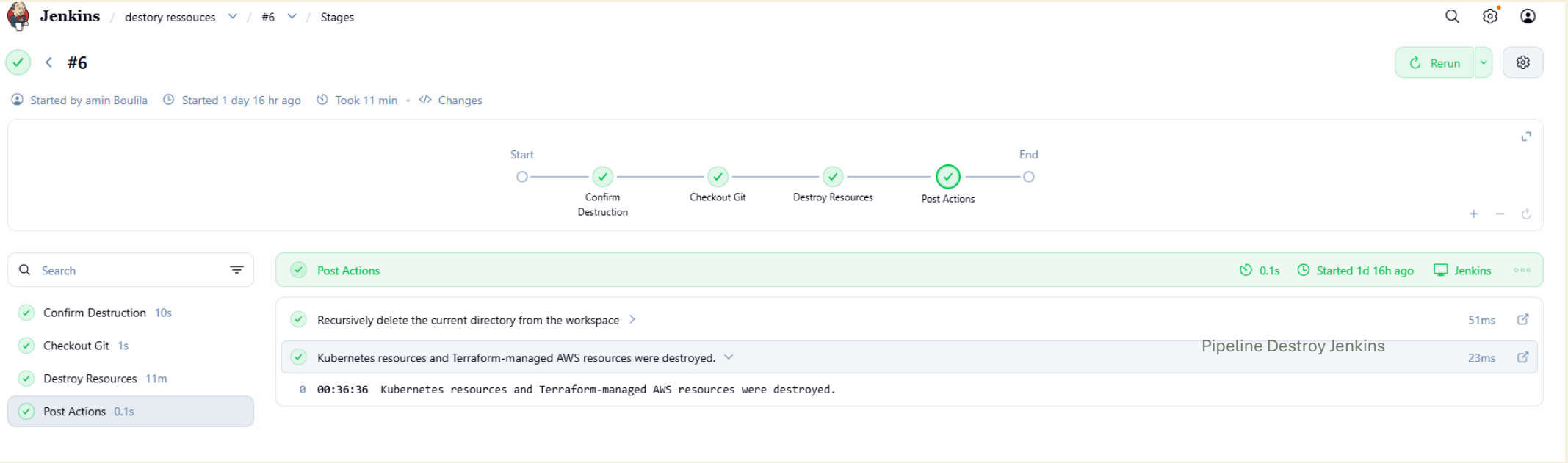


Déclenchement webhook dans Jenkins

Suppression contrôlée de l'infrastructure

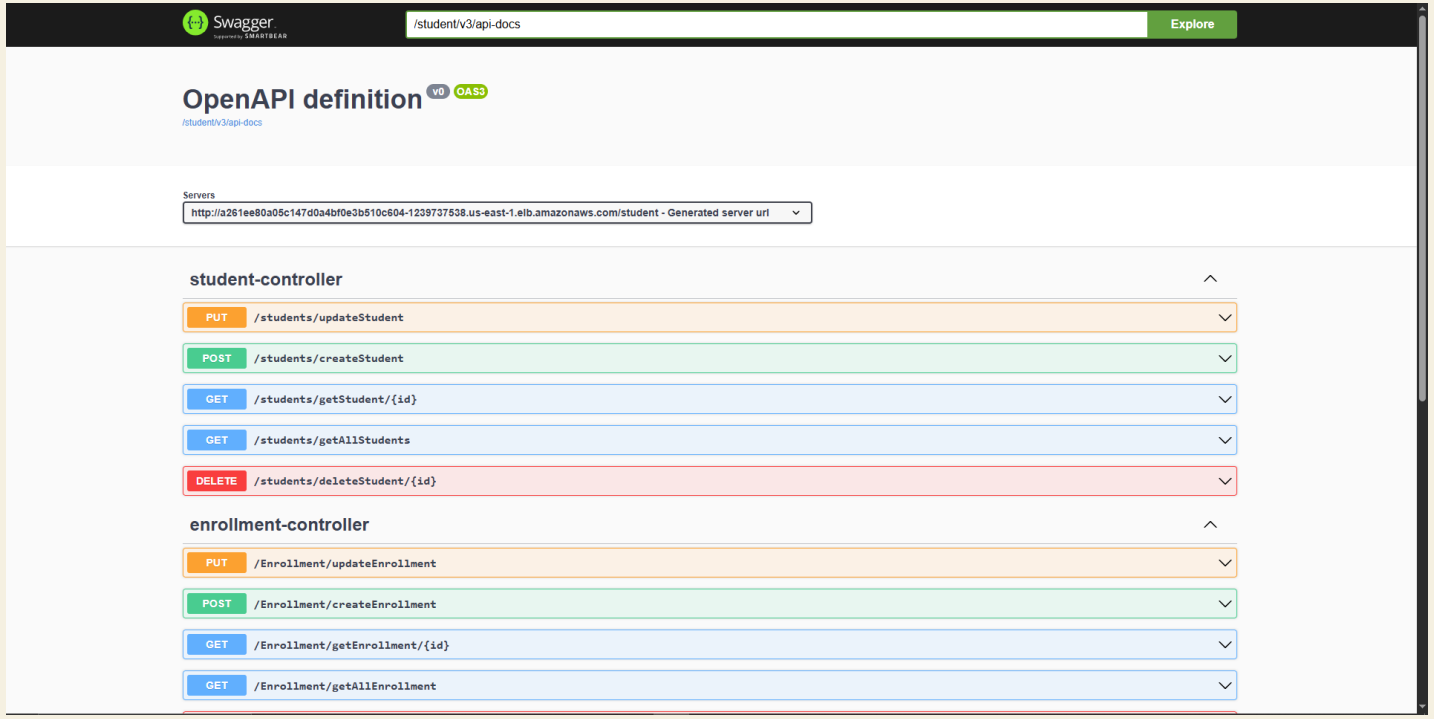
```
                input(message: 'Delete the Kubernetes resources and all Terraform-managed AWS infrastructure?')
aws sts get-caller-identity
terraform -chdir=terraform init -reconfigure -input=false
aws eks update-kubeconfig --region "$AWS_DEFAULT_REGION" --name "$EKS_CLUSTER_NAME"
kubectl delete -f kubernetes/ --ignore-not-found=true --wait=true
aws eks delete-addon --cluster-name "$EKS_CLUSTER_NAME" \
  --addon-name aws-ebs-csi-driver
terraform -chdir=terraform destroy -input=false -auto-approve \
  -var="cluster_role_arn=$EKS_ROLE_ARN" \
  -var="node_role_arn=$EKS_ROLE_ARN"
```

Garde-fou : validation manuelle avant suppression • post : deleteDir()



Pipeline Destroy Jenkins

Application déployée et accessible



Résultat applicatif

Résultat

- Build Maven + tests
- Qualité SonarQube
- Artefact Nexus
- Image Docker Hub
- EKS + MySQL
- Monitoring opérationnel

<LOAD_BALANCER_DNS>

Difficultés rencontrées et améliorations possibles

Difficultés

Coordination des credentials Jenkins et des secrets

Création puis accès au cluster EKS

Persistance MySQL via EBS CSI

Exposition Jenkins local via ngrok

Synchronisation pipeline / disponibilité du LoadBalancer

Améliorations

Secrets manager AWS ou Vault

Domaine personnalisé + TLS

Images versionnées et déploiement par digest

Environnements dev / staging / prod

Tests d'intégration et scans de vulnérabilités

HPA, autoscaling et alertes SLO

Conclusion et remerciements

Une chaîne reproductible : du commit à l'application monitorée sur AWS EKS.

Merci pour votre attention

Les commandes d'installation ont été ajoutées au support.
Les secrets restent masqués et les preuves visuelles sont conservées à leur ratio d'origine.

Captures manquantes : aucune